



BOOTCAMP FRONTEND & REACT · SEMANA 4

Los datos de verdad

Clase 10 — fetch, promesas, async/await y manejo de errores: TechCart le pide sus productos a un servidor

Semana 4 · L·M·V

Proyecto TechCart

STEERCLAVE

LOS SEIS PRODUCTOS LOS ESCRIBÍ YO · HOY LOS TRAE UN SERVIDOR

Agenda

1. La API y el JSON

de dónde salen los datos de una tienda

3. Promesas

tres estados · then · catch · finally

5. async / await

la misma promesa, de corrido

7. Manejo de errores

try · catch · finally · cancelar

2. Por qué asíncrono

un solo hilo, y lo que tarda la red

4. fetch

la respuesta, y la trampa del 404

6. El adaptador y Promise.all

api.js: el archivo nuevo

8. Los cuatro estados y cierre

cargando · éxito · vacío · error



Receso de 15 min después de **fetch**, antes de escribir el código del día

Dos por el chat

¿Por qué `pintarCatalogo` recibe la lista por parámetro en vez de usar el arreglo directo?

¿Por qué el listener va en el contenedor y no en cada botón?

UNA DIRECCIÓN QUE DEVUELVE DATOS EN VEZ DE UNA PÁGINA

Qué es una API

la página

dibuja tarjetas,
botones y precios

→ pide

la API

devuelve datos
crudos,
sin ningún adorno

← responde

el servidor

es el que sabe
qué hay en stock

DummyJSON

pública y gratis, sin registro ni
clave. Hecha para practicar.

194 productos

repartidos en 24 categorías, de
maquillaje a motos.

TechCart vende 4

laptops · smartphones · tablets ·
mobile-accessories = **38**

TODO LO QUE VIAJA POR INTERNET VIAJA COMO TEXTO

JSON no es un objeto de JavaScript

OBJETO DE JAVASCRIPT

```
const p = {  
  nombre: 'iPhone',  
  precio: 999,  
  mostrar() { ... },  
};
```

≠

JSON

```
{  
  "nombre": "iPhone",  
  "precio": 999  
}
```

comillas dobles

en las claves y en los textos,
siempre

sin funciones

ni comentarios, ni undefined

sin coma final

la última no lleva coma

no es de JS

Java, Python y Go contestan
igual

la respuesta viene envuelta: { products, total, skip, limit } • el arreglo está dentro de products

PINTAR, RESPONDER AL CLIC, HACER SCROLL: TODO EN LA MISMA FILA

JavaScript tiene **un solo hilo**

si esperara...

la fila se detiene en la petición. No responden los clics, no hay scroll, no se puede seleccionar texto. El navegador pregunta si quieres cerrar la pestaña.

no espera

manda el pedido, lo deja encargado y sigue con lo que tenga. Cuando la respuesta llega, el trabajo pendiente se pone al final de la fila.

Eso es ser asíncrono: la operación empieza en un momento y termina en otro, y en el medio el programa hace otras cosas. El mecanismo que lo ordena se llama **bucle de eventos**. La consecuencia, que es lo que hay que llevarse: **lo que espera no se hace en el momento, se hace después.**

POR QUÉ ESTO EXISTE SOLO PARA LA RED

Lo que tarda **la red**

~ 0 ms

leer una variable, sumar dos números, recorrer un arreglo de 38

16 ms

pintar un cuadro de animación: el límite de lo que el ojo nota

100 - 2000 ms

pedirle algo a un servidor, y más si la red está mal

Nada de lo que hicimos hasta ayer necesitaba esperar. El catálogo se dibujaba y aparecía; el carrito sumaba y aparecía. Todo pasaba dentro de la máquina. Lo de hoy cambia en una sola cosa, y lo cambia todo: hay que **salir** de la máquina.

🔗 EN JAVA LO TIENEN Y SE LLAMA COMPLETABLEFUTURE

Un **vale** por un resultado que todavía no llegó

pendiente

mientras espera. Es como nace toda promesa.

cumplida

llegó. Trae el **valor** adentro.

rechazada

falló. Trae el **error** adentro.

de pendiente se sale UNA vez y para siempre

Por eso una promesa **no** sirve para lo que pasa muchas veces, como los clics; para eso están los eventos de la clase pasada. Una promesa es para **un** resultado.

THEN DEVUELVE OTRA PROMESA, Y POR ESO SE ENCADENAN

then • catch • finally

.then(valor => ...)

cuando se cumpla, haz esto, con el valor que traiga

.catch(error => ...)

si se rechaza, haz esto otro, con el error que traiga

.finally(() => ...)

pase lo que pase: apagar el indicador, volver a habilitar el botón

```
fetch(url)
  .then((respuesta) => respuesta.json())
  .then((datos) => usar(datos))
  .catch((error) => avisar(error.message))
  .finally(() => apagarCargando());
```

QUÉ PROBLEMA VINIERON A RESOLVER LAS PROMESAS

Antes: los **callbacks**

CALLBACKS ANIDADOS

```
pedir(a, (x) => {  
  pedir(b, (y) => {  
    pedir(c, (z) => {  
      ...  
    })  
  })  
})
```

cada paso adentro del anterior, y el error se maneja **tres** veces



PROMESAS ENCADENADAS

```
pedir(a)  
  .then(...)  
  .then(...)  
  .then(...)  
  .catch(...)
```

los pasos en fila, y **un solo catch** al final atrapa el fallo de cualquiera

hoy no escribimos callbacks: solo hay que saber de dónde venimos

VIENE CON EL NAVEGADOR: NO HAY QUE INSTALAR NADA

fetch devuelve una promesa, no los datos

```
// en la consola, sin esperar nada  
console.log(fetch(url)); → Promise { <pending> }
```

respuesta.ok

true cuando el estado va de 200 a 299

respuesta.status

200, 404, 500: el número que contestó el servidor

respuesta.headers

las cabeceras: entre ellas, de qué tipo es el contenido

la respuesta es el sobre • la carta se lee aparte, con .json()

fetch NO rechaza con un 404 ni con un 500

la promesa se RECHAZA

no hay internet · el dominio no existe · se cortó la conexión · la petición se canceló

solo cuando no se pudo **hablar** con el servidor

la promesa se CUMPLE

200 · 301 · **404** · **500**

hubo comunicación: el servidor escuchó, entendió y contestó. Para fetch, eso es éxito

```
// esta línea es NUESTRA: fetch no la escribe por nosotros  
if (!respuesta.ok) throw new Error('El servidor respondió ${respuesta.status}');
```

POR ESO UN FETCH LLEVA DOS ESPERAS, NO UNA

El sobre, y después **la carta**

`await fetch(url)`

llega el **sobre**:
ok, status, headers



`await
respuesta.json()`

se lee la **carta**:
recién aquí están los datos

¿Por qué `.json()` también tarda? Porque el cuerpo puede ser enorme y, cuando llegó el sobre, puede que todavía esté viajando. Hace lo mismo que `JSON.parse` con el `localStorage` de la clase pasada: convierte **texto** en objetos. Y como tarda, también devuelve una promesa.

también existen `.text()` y `.blob()` • el cuerpo se lee una sola vez

ES LA MISMA PROMESA: NO HAY NADA NUEVO POR DEBAJO

async / await

CON THEN

```
fetch(url)
  .then((r) => r.json())
  .then((d) => usar(d))
```

=

CON AWAIT

```
const r = await fetch(url);
const d = await r.json();
usar(d);
```

 **Java:** await se parece al `get()` de un `Future`, con una diferencia enorme: **no bloquea el hilo**. Mientras esta función espera, el navegador sigue atendiendo clics y scroll.

CON ESTAS TRES QUEDAN CUBIERTOS

Las tres reglas de **async**

1 · dónde vive await

solo dentro de una función **async**.
Excepción que a nosotros sí nos toca: en el **nivel superior de un módulo** también se puede.

2 · qué devuelve **async**

una **promesa**, siempre. Aunque dentro devuelvan el número 5.

3 · la consecuencia

si llaman a una función **async** y quieren su resultado, también hay que **esperarla**.

```
const dameCinco = async () => 5;  
console.log(dameCinco());           → Promise {<fulfilled>: 5}  
console.log(await dameCinco()); → 5
```

🔗 EN JAVA TIENE NOMBRE PROPIO: EL PATRÓN ADAPTER

Un traductor en la puerta



Por qué no es burocracia. Si toda la tienda leyera p.thumbnail directo, el día que la API cambie ese nombre hay que corregir todos los archivos y rezar que no falte ninguno. Con el adaptador se corrige **una línea**. Eso es **desacoplar**: que una pieza cambie sin arrastrar a las demás.

lo que la API no trae lo decide el adaptador: alt sale del título y destacado es una decisión de la tienda

EL CASO QUE NO CUBRE NINGUNA OTRA COSA: LA RED QUE NO FALLA Y NO RESPONDE

AbortController: cortar una petición

```
// la forma completa: cancelar cuando uno quiera
const control = new AbortController();
fetch(url, { signal: control.signal });
control.abort(); // p. ej. al lanzar la búsqueda siguiente
```

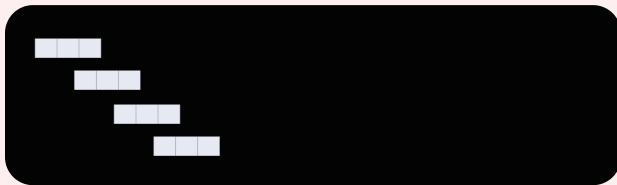
```
// la forma corta, la que usamos hoy: cancelar por tiempo
fetch(url, { signal: AbortSignal.timeout(8000) });
```

Sin esto, una petición que sale y nunca vuelve deja la promesa **pendiente para siempre**, y la página con el "Cargando..." puesto. El error que llega al cancelar se llama **TimeoutError**, y merece su propio mensaje.

CUATRO CATEGORÍAS = CUATRO PEDIDOS QUE NO DEPENDEN UNOS DE OTROS

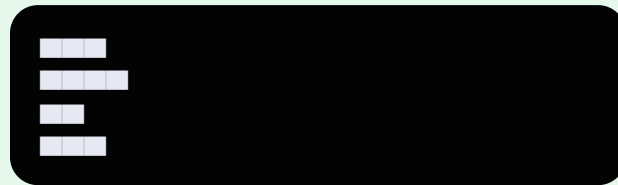
Promise.all: todos a la vez

en escalera



cada await espera al anterior: el total es la **suma**.
Medido hoy: **1425 ms**

a la vez



salen juntas: el total es la **más lenta**. Medido hoy: **291 ms**

```
const listas = await Promise.all(categorias.map(pedirCategoria));
return listas.flat(); // llega un arreglo de arreglos: flat lo aplana
```

si una falla, falla el conjunto · para "lo que se pueda" está Promise.allSettled

EL MISMO DE JAVA, Y CON AWAIT ATRAPA LAS PROMESAS RECHAZADAS

try / catch / finally

try

lo que puede fallar va aquí dentro

catch (error)

recibe el error y, en vez de dejar la página muerta, le habla a la persona

finally

corre **siempre**: aquí se apaga lo que se encendió al empezar

Qué atrapa y qué no. Con `await` delante, si la promesa se rechaza el `await` **lanza** y el `catch` se entera. Pero si llaman a una función que devuelve una promesa y **no la esperan**, el fallo ocurre después, fuera del bloque, y ese `catch` ya pasó: no lo atrapa nadie. En la consola aparece `Uncaught (in promise)`.

la forma más común de que una página parezca colgada: un "Cargando..." que nadie apagó

CAEN EN EL MISMO CATCH, PERO LA PERSONA PUEDE HACER COSAS DISTINTAS

Tres fallos **distintos**

```
TypeError: Failed to fetch
```

Error de red. No hay internet, el dominio no responde, la petición se canceló. Es el **único** caso en que fetch rechaza solo.

```
if (!respuesta.ok) throw ...
```

Respuesta no correcta. 404, 500. Fetch la da por buena, así que el error lo lanzamos nosotros.

```
if (!Array.isArray(datos.products))
```

Datos inesperados. Estado 200 y adentro no vino la lista. Tampoco se queja nadie: lo miramos nosotros.

A la **pantalla** va lo que la persona puede hacer ("revisa tu conexión y vuelve a intentarlo"). A la **consola** va el detalle técnico, para quien mantiene el código. Los dos salen del mismo error.

PEDIR DATOS SON CUATRO CAMINOS, NO UNO

Cargando · éxito · vacío · error

cargando

esqueletos del tamaño de las tarjetas que van a venir, para que la página no dé un salto

éxito

el catálogo, y cuántos productos llegaron de cada categoría

vacío

200, todo bien, y adentro no había nada. No es un error: no lleva botón

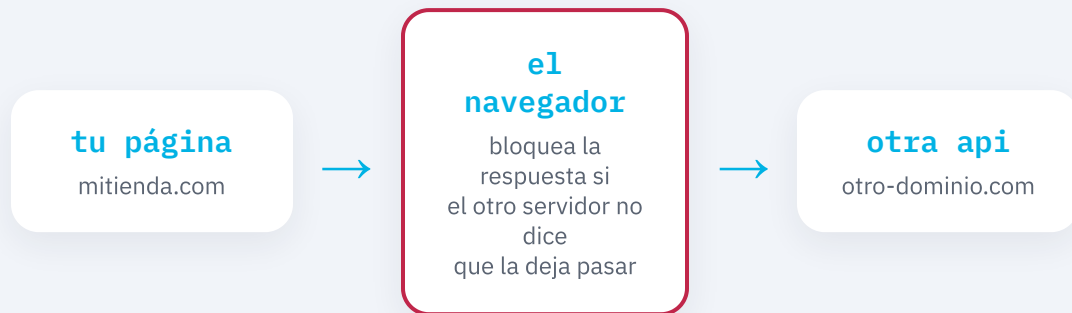
error

el mensaje en rojo y un botón de **Reintentar**, porque un error con arreglo no es un error

Una **página en blanco no se lee como "espera"**: se lee como "esto está roto", y la gente se va. Y la página tiene que seguir viva: falló el catálogo, no el carrito ni el formulario.

FUNCIONA EN LA BARRA DE DIRECCIONES Y FALLA DESDE TU PÁGINA

CORS, en una línea



No es un error de su código y no se arregla desde el frontend. Lo habilita el **servidor**, con una cabecera. Cuando pegan la dirección en la barra no hay ninguna página de otro dominio recibiendo la respuesta, por eso ahí funciona. DummyJSON lo tiene abierto, y por eso hoy no apareció.

TODO LO QUE LLEGA AL NAVEGADOR ES PÚBLICO

La clave de una API **nunca** va en el frontend

en el código

```
const CLAVE = "sk_live_7f3a...";
```

cualquiera la lee en la pestaña Sources de DevTools. No está escondida: está **publicada**. Y suele estar atada a una tarjeta de crédito.

en un servidor propio

```
pagina → mi servidor → la API
```

el servidor guarda la clave y hace la petición. El navegador nunca la ve.

la regla, desde el primer día: si es secreto, no va en el frontend

LOS MENSAJES SON LOS REALES, REPRODUCIDOS EN EL NAVEGADOR

Errores frecuentes

```
Cannot read properties of undefined  
(reading 'map')
```

1 • creer que fetch lanza con un 404. No lanza: el castigo llega dos pasos después. Si la ruta devuelve una página de error, el que falla es `.json()` con Unexpected token '<'

```
r.json is not a function
```

2 • olvidar el await y tratar la promesa como el dato. Con nuestras funciones es `lista.map is not a function`

```
Cannot read properties of undefined  
(reading 'length')
```

3 • olvidar que `response.json()` también se espera. Son **dos** await, siempre

```
SyntaxError: await is only valid in async  
functions...
```

4 • await fuera de una función async. No es un error al correr: el archivo entero no se ejecuta

```
Uncaught (in promise) TypeError: Failed to  
fetch
```

5 • no manejar el error: esqueletos latiendo para siempre, y la tienda parece colgada

```
(sin error: solo lentitud)
```

6 • pedir en un bucle con await. Se ve en escalera en la pestaña Red, y se arregla con `Promise.all`

```
(sin error: la tarjeta dice undefined)
```

7 • asumir que el campo se llama como el nuestro. `p.titel` y `p.nombre` dan undefined, sin avisar

LO QUE TECHCART SABE HACER DESDE HOY

Resumen

un solo hilo

por eso lo lento no puede esperar en la fila

Promise

pendiente · cumplida · rechazada

then · catch · finally

se encadenan porque then devuelve otra promesa

fetch

devuelve una promesa; llega primero el sobre

ok y status

un 404 NO rechaza: el throw es nuestro

.json()

también se espera: son dos await

async / await

la misma promesa, de arriba abajo

async devuelve promesa

siempre, aunque dentro devuelvan un número

Promise.all

para lo que no depende de nada más

try · catch · finally

tres fallos distintos, tres mensajes

AbortSignal

lo que no contesta se cancela

el adaptador

el único archivo que habla los dos idiomas

y las dos de llevarse: el adaptador desacopla · pedir datos son cuatro estados, no uno

LA FÁCIL HÁGANLA TODOS: ES REPLICAR LA CLASE

5 tareas, en tres niveles

FÁCIL

Replicar la clase, y que el botón avise. La tienda como quedó hoy, comprobada con la red lenta y en Offline. Más: que "Reintentar" se apague y diga "Reintentando..." mientras pide, y vuelva en el finally.

INTERM. 1

Que una categoría caída no deje la tienda vacía. Cambiar `Promise.all` por `Promise.allSettled`, juntar lo que sí llegó y decir en el estado cuántas faltaron.

INTERM. 2

El detalle del producto. Clic en el nombre → pedir `/products/{id}` y mostrar la descripción. Con su propio ok y sus propios estados. Un id que no existe da 404: perfecto para probarlo.

DIFÍCIL 1

No pedir dos veces lo mismo. Guardar lo que llegó en `sessionStorage` (con try/catch), avisar que son datos guardados y ofrecer un botón de "Actualizar". Eso es una **caché**.

DIFÍCIL 2

El buscador que le pregunta al servidor. Evento input, **debounce** para no pedir en cada tecla, y **AbortController** de verdad para cancelar la búsqueda anterior. Si no cancelan, una respuesta vieja pisa a la nueva.

Próxima clase · último día de JavaScript

La tienda **terminada**

Con 38 productos en pantalla, **filtrar por categoría** y **buscar** dejaron de ser un lujo. Y con ellos llega la idea que ordena todo lo que viene: hay **datos**, y lo que se ve se **calcula** a partir de ellos. Más el carrito con **cantidades** y el pedido completo.

Hoy escribí **p.title**. ¿Y si me equivoco y escribo **p.title**?

Traigan la tienda funcionando con los 38 productos. Nos vemos.